

Orçamento Farmacêutico Híbrido

Pipeline de **RAG** (Retrieval-Augmented Generation) que lê um e-mail de solicitação, encontra os medicamentos no catálogo usando **busca híbrida** (semântica + lexical) e devolve um orçamento estruturado em JSON gerado por LLM.

n8n · orquestração

LM Studio · embeddings + LLM local

Supabase / Postgres · pgvector

bge-m3 · 1024 dimensões

RRF · Reciprocal Rank Fusion

981 produtos

Índice

1. Visão geral da arquitetura
2. Como construímos os chunks
3. Como construímos o banco de dados
4. Ingestão: gerando os embeddings
5. O fluxo de consulta (n8n)
6. Busca híbrida + RRF
7. Geração do orçamento (LLM)
8. Componentes e portas

1 Visão geral da arquitetura

O sistema tem duas fases bem distintas. A **fase offline** (feita uma vez) prepara o conhecimento: transforma um CSV de catálogo em pedaços de texto pesquisáveis (*chunks*) e os carrega no banco vetorial. A **fase online** (a cada e-mail) consulta esse banco e monta o orçamento.

Fase offline — indexação

- `main.py` → CSV vira **chunks** JSON
- `setup.sql` → cria a tabela `documents` + pgvector
- `ingest.py` → gera **embeddings** e popula o banco

Roda uma vez (ou quando o catálogo muda).

Fase online — consulta

- Workflow n8n recebe o e-mail do cliente
- Busca **semântica + lexical** em paralelo
- Funde resultados com **RRF** e chama a LLM
- Devolve o **orçamento em JSON**

Roda a cada solicitação de orçamento.

Por que "híbrido"? A busca semântica entende *significado* (ex.: "remédio para herpes" acha "ACICLOVIR"), mas erra em códigos e nomes exatos. A busca lexical acerta o código SAP e o texto literal, mas não entende sinônimos. Combinar as duas via RRF dá o melhor dos dois mundos.

2 Como construímos os chunks

Script: `main.py`. Ele baixa o CSV oficial do catálogo (separado por `;`, padrão de arquivo fiscal) e converte **cada linha de produto em um "chunk"** — um objeto pronto para indexação. São **981 produtos** no total.

Cada chunk tem três partes, e a separação entre elas é o ponto-chave do desenho:

Campo	Papel	Conteúdo
<code>id</code>	Chave única	<code>sap_1006459</code> — prefixo <code>sap_</code> + código SAP
<code>page_content</code>	Texto que vira embedding	Frase legível: apresentação, princípio ativo, código SAP/EAN, família, tipo
<code>metadata</code>	Payload "oculto" p/ cálculo	Todos os preços por faixa de ICMS (PF e PMC), PIS, COFINS, NCM, etc.

```
# main.py — o texto que a LLM/embedding vai "ler" para achar o produto
page_content = f"Apresentação: {apresentacao}. Princípio Ativo: {principio_ativo}. "
               f"Código SAP: {codigo_sap}. Código EAN: {ean}. "
               f"Família: {familia}. Tipo: {tipo_med}."

# O payload "oculto" que o n8n usa depois para a matemática do preço
metadata = {
    "codigo_sap": codigo_sap,
    "apresentacao": apresentacao,
    "icms_17_pf": to_float(row.get('ICMS 17 % (PF)'),), # preço Pessoa Física
    "icms_17_pmc": to_float(row.get('ICMS 17 % (PMC)'),), # preço Máximo ao Consumidor
    # ... faixas 0/12/17,5/18/20 %, áreas de livre comércio, PIS, COFINS
}
```

A ideia central: só o `page_content` é vetorizado e usado para *encontrar* o produto. Os preços ficam no `metadata` — eles não poluem a busca semântica, mas estão à mão na hora de *calcular* o orçamento. Cada texto é curto e autocontido (1 produto = 1 chunk), o que dá embeddings precisos e evita "diluição" de contexto.

Saída: `chunks_hibridos_catalogo_completo.json` — uma lista com os 981 objetos acima.

3 Como construímos o banco de dados

Script: `setup.sql` (rodado uma vez no SQL Editor do Supabase). Ele liga a extensão **pgvector** e cria a tabela `documents`, que é o coração do retrieval.

```

create extension if not exists vector;           -- habilita o tipo vector no Postgres

create table documents (
  id          bigserial primary key,
  chunk_id    text unique,                    -- = id do chunk (sap_xxxx)
  content      text not null,                 -- = page_content
  metadata     jsonb not null,                -- = metadata (preços!)
  embedding    vector(1024)                  -- vetor do bge-m3
);

-- Índice ANN para busca semântica rápida por distância de cosseno
create index documents_embedding_hnsw
  on documents using hnsw (embedding vector_cosine_ops);

-- Índice no código SAP dentro do JSON, para a busca lexical exata
create index documents_metadata_codigo_sap
  on documents ((metadata->'codigo_sap'));

```

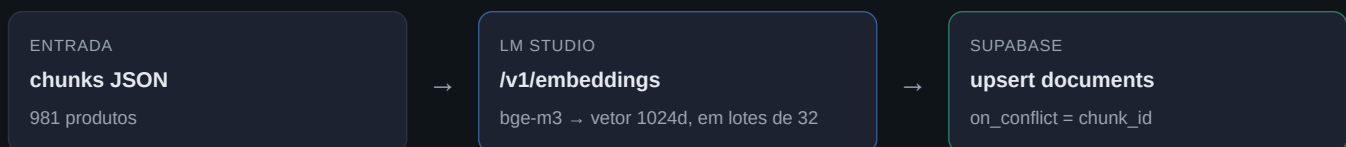
Dois índices, um para cada modo de busca:

- **HNSW** (`vector_cosine_ops`) — acelera o vizinho-mais-próximo aproximado usado na busca *semântica*.
- **Índice em** `metadata->'codigo_sap'` — acelera o filtro exato por código usado na busca *lexical*.

O arquivo ainda cria a função `match_documents(...)` (helper opcional do padrão n8n Vector Store); o workflow atual, porém, monta as queries SQL à mão para controlar o híbrido — ver seção 6.

4 Ingestão: gerando os embeddings

Script: `ingest.py`. Ele lê o JSON de chunks, pede a **LM Studio** (servidor local com o modelo `bge-m3`) que transforme cada `page_content` em um vetor de 1024 dimensões, e faz `upsert` em lote na tabela `documents`.



```

# ingest.py - laço principal (resumido)
for start in range(0, len(chunks), batch_size):      # BATCH_SIZE=32
    batch = chunks[start:start+batch_size]
    texts = [c["page_content"] for c in batch]
    vectors = embed_batch(http, base_url, model, texts) # chama LM Studio

    rows = [{
        "chunk_id": c["id"],
        "content": c["page_content"],
        "metadata": c["metadata"],
        "embedding": vec,
    } for c, vec in zip(batch, vectors)]

    supabase.table(table).upsert(rows, on_conflict="chunk_id").execute()

```

Sanity check de dimensão: antes de processar tudo, o script vetoriza *um* texto e confere se o modelo devolve exatamente `EMBEDDING_DIM` (1024). Se não bater com o `vector(1024)` do SQL, ele aborta — evita popular o banco com vetores do tamanho errado. O `upsert` por `chunk_id` torna a ingestão **idempotente**: rodar de novo atualiza em vez de duplicar.

5 O fluxo de consulta (workflow n8n)

Arquivo: `Orçamento Farmaceutico Híbrido.json`. É o workflow do n8n que roda a cada solicitação. Visão de ponta a ponta:



Exemplo de e-mail de entrada

```
Ola, gostaria de receber um orcamento:
10x ACICLOVIR 200MG COM BLX25
10x BIOMAG 15MG CAP BLX30

Jose Farmacia - Rio de Janeiro - RJ
```

6 Busca híbrida + RRF

O node **"Preparar Buscas"** recebe o embedding do e-mail e monta as duas queries que rodam em paralelo contra a mesma tabela `documents`:

Semântica (vetorial)

```
SELECT chunk_id, content, metadata,  
       1 - (embedding ⇔ '[...vetor...]') AS score  
FROM documents  
ORDER BY embedding ⇔ '[...vetor...]'  
LIMIT 20;
```

Ordena por proximidade de cosseno (⇔). Acha produtos por significado.

Lexical (texto/código)

```
SELECT chunk_id, content, metadata, 1.0 AS score  
FROM documents  
WHERE metadata→'codigo_sap' = '1006459'  
      OR content ILIKE '%ACICLOVIR%'  
LIMIT 20;
```

O node extrai **códigos** (6–7 dígitos) e **palavras-chave** do e-mail, removendo *stopwords* (para, com, orçamento...).

Fusão por RRF (Reciprocal Rank Fusion)

O node **"Aplicar RRF e Montar Prompt"** combina as duas listas. Cada documento ganha pontos pela *posição* (rank) que ocupou em cada busca — não pelo score bruto, o que evita ter de normalizar escalas diferentes (cosseno vs. ILIKE):

```
const K = 60; // constante padrão do RRF  
  
// para cada documento, em cada lista, soma 1 / (K + posição)  
score[id] += 1 / (K + idx + 1);  
  
// ordena por score combinado e fica com os 10 melhores  
topDocs = Object.entries(scores).sort((a,b) => b[1] - a[1]).slice(0, 10);
```

Um produto que aparece *bem colocado nas duas* buscas sobe ao topo — que é exatamente o sinal de maior confiança. Os 10 vencedores, já com seus preços (`metadata`), formam o "catálogo relevante" entregue à LLM.

7 Geração do orçamento (LLM)

O mesmo node monta um prompt com (a) o e-mail original e (b) os 10 produtos top-RRF formatados com todas as faixas de preço. O node **"Chamar LM Studio"** envia isso a um modelo de chat local (`gemma-3n`) com instruções precisas:

- Casar o que o cliente pediu com os produtos do catálogo
- Extrair **quantidades** (padrão 1 se não informado)
- Identificar **tipo de cliente** (PF, PJ ou Revenda) e **UF**
- Escolher o preço: PF/PJ → `icms_17_pf`; Revenda → `icms_17_pmc`
- Calcular `preço_total = quantidade × preço_unitário` e o total geral
- Responder **apenas com JSON válido**, sem markdown

Por fim, o node **"Extrair JSON do Orçamento"** limpa eventuais cercas de código e faz `JSON.parse`, devolvendo a estrutura final:

```
{
  "itens": [
    { "produto": "ACICLOVIR 200MG COM BLX25", "codigo": "1006459",
      "quantidade": 10, "preco_unitario": 93.67, "preco_total": 936.70 },
    { "produto": "BIOMAG 15MG CAP BLX30", "codigo": "1002589",
      "quantidade": 10, "preco_unitario": 35.54, "preco_total": 355.40 }
  ],
  "cliente": { "tipo": "PJ", "uf": "RJ" },
  "total": 1292.10
}
```

Resumo do "porquê": a LLM nunca inventa preços — ela só escolhe entre os valores reais que vieram no `metadata` dos produtos recuperados. O retrieval garante *dados corretos*; a LLM faz a *interpretação* e a *aritmética*. É isso que torna o resultado confiável.

8 Componentes e portas

Componente	Papel	Endereço / modelo
n8n	Orquestra todo o fluxo online	workflow <code>Lv0FvpRryNgAffjK</code>
LM Studio — embeddings	Vetoriza textos (offline e query)	<code>:1234/v1/embeddings</code> · <code>bge-m3</code> (1024d)
LM Studio — chat	Gera o orçamento JSON	<code>:1234/v1/chat/completions</code> · <code>gemma-3n</code>
Supabase / Postgres	Banco vetorial + lexical	tabela <code>documents</code> · pgvector

Tudo local: embeddings e LLM rodam na própria máquina via LM Studio (`172.17.0.1:1234` = host visto de dentro do container do n8n). Nenhum dado de catálogo ou de cliente sai para serviços externos.